

Information Retrieval 10

Bi-Encoders and Dense Retrieval

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following textbook:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

Vocabulary mismatch problem: When queries and documents use different terms, keyword-based retrieval cannot find relevant documents

Document expansion (doc2query): Generate anticipated queries from documents and append them. Processed at indexing time with no additional cost at search time

DeepCT / DeepImpact: Learn contextual importance of terms using BERT, improving term weights in the inverted index

This time we learn an entirely new approach that does not rely on vocabulary: retrieval using **Dense Vectors**

Understand the principles and major methods of **Dense Retrieval**, which converts queries and documents into dense vectors and performs fast retrieval through nearest neighbor search in vector space

First Half: Foundations of Dense Retrieval

Limitations of Cross-encoders and Bi-encoders

Approximate Nearest Neighbor (ANN) Search

Sentence-BERT: Learning Sentence Embeddings

Second Half: Major Dense Retrieval Methods

DPR: Dense Passage Retrieval

ANCE: Dynamic Hard Negative Mining

ColBERT: Late Interaction Model

Design Space of Bi-encoders

Limitations of Cross-encoders and Bi-encoders

Why Cross-encoders Cannot Be Used for Retrieval

Let us review the progression of models covered so far:

1. **Representation-based:** DSSM — Independently vectorizes query and document. Weak interaction
2. **Interaction-based:** DRMM — Uses similarity matrices but without context
3. **Cross-encoder:** monoBERT — **Concatenates** query and document as input to BERT. High accuracy but slow

Next step: A model that maintains high accuracy while being **usable for retrieval (first stage)** is needed

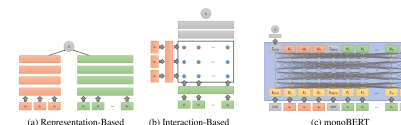


Figure 22. The evolution of neural models for text ranking, copied from Figure 11 in Section 3.2.3: representation-based approaches (left), interaction-based approaches (middle), and BERT (right). Dense representations for ranking are most similar to representation-based approaches, except that more powerful transformer-based encoders are used to model queries and texts from the corpus.

- the comparison problem, or the design of ϕ , which involves a balance between what can be efficiently computed at scale and what is necessary to capture relevance in terms of the dense representations.

As we'll discuss in Section 5.3, both challenges predate BERT, although transformers broaden the design space of η and ϕ .

The complete model comprised of η_q , η_d , and ϕ is usually developed in a supervised manner. In the transformer context, the encoders (and ϕ in some cases as well) are trained (or fine-tuned, to be more accurate) with labeled data capturing the target task. The outputs of η_q and η_d in the supervised scenario are called learned representations, and thus the problem formulation is an instance of representation learning. In principle, the encoders may have never been exposed to labeled training data for the target task. When using pretrained transformers, however, the models may have been exposed to the target corpus during pretraining, but it seems odd to call the encoders "unsupervised" in this context. More common is the case where the models are fine-tuned on out-of-distribution data (e.g., different queries, different corpora, or both) and directly applied to previously unseen texts in a "zero-shot" manner.

Another way to think about dense representations for ranking is in terms of the evolution of broad classes of neural ranking models, dating back to pre-BERT approaches discussed in Section 1.2.4. A side-by-side comparison between pre-BERT representation-based models, pre-BERT interaction-based models, and BERT is shown in Figure 11 in Section 3.2.3 and repeated here as Figure 22. The dense retrieval approaches we focus on in this section are architecturally similar to representation-based approaches, Figure 22(a), except that more powerful transformer-based encoders are used to model queries and texts from the corpus. In previous models, the "arms" of the network that generate the vector representations (i.e., the encoders) are based on CNNs or RNNs. Today, these have been replaced with BERT and other transformers. For the choice of the comparison function, pre-BERT representation-based neural ranking models adopt a simple ϕ such as inner product. With transformer-based representations, such simple comparison functions remain common. However, researchers have also explored more complex formulations of ϕ , as we will see in Section 5.5. Some of these approaches incorporate interactions between terms in the queries and texts from the corpus, reminiscent of pre-BERT interaction-based models.

What are the motivations for exploring this formulation of the text ranking problem? We can point to two main reasons:

- BERT inference is slow. This fact, as well as potential solutions, was detailed in Section 3.5. The formulation of text ranking in terms of $\phi(\eta_q(q), \eta_d(d))$ has two key properties: First, note that $\eta_d(d)$ is not dependent on queries. This means that text representations can be precomputed and stored, thus pushing potentially expensive neural network inference into a preprocessing stage—similar to doc2query and DeepCT (see Section 4). Although $\eta_q(q)$ still needs to be computed at query time, only a single inference is required, and over a relatively short sequence of tokens (since queries are usually much shorter than texts from the corpus).

Source: Lin et al. (2021), Figure 22. Evolution from representation-based to interaction-based to BERT-based models

Cross-encoders (monoBERT, etc.) concatenate query q and document d as input to BERT:

input = [CLS] q [SEP] d [SEP]

Problem: BERT must be executed for every query-document pair

Concrete computational cost:

Collection: 8.8 million documents (MS MARCO)

Inference per pair: ~30ms (GPU)

Per query: 8.8M x 30ms = ~73 hours

-> Cannot be used for first-stage retrieval. Only usable for reranking after narrowing candidates with BM25, etc.

We begin by illustrating the basic design of bi-encoders with Sentence-BERT [Reimers and Gurevych, 2019] in Section 5.4.1. Sentence-BERT, however, focused on sentence-similarity tasks and did not specifically tackle retrieval problems. In Section 5.4.2, we present DPR [Karpukhin et al., 2020b] and ANCE [Xiong et al., 2021] as exemplary instances of dense retrieval implementations built on the basic bi-encoder design. Additional bi-encoder variants that help us better understand the design space and key research issues are discussed in Section 5.4.3.

Before getting started, however, we present some historical background on the development of dense retrieval techniques in order to recognize precedence and the important contributions of many researchers. Since our overall presentation does not necessarily focus on the earliest known work, we feel it is important to explicitly acknowledge how these ideas evolved.

Transformer-based dense representations for semantic equivalence, paraphrase, entailment, and other sentence-similarity tasks can be traced back to the Universal Sentence Encoder (USE) [Cer et al., 2018a,b], which dates to March 2018, even before BERT was introduced! The Universal Sentence Encoder aspired to be just that: to encode “sentences into embedding vectors that specifically target transfer learning to other NLP tasks”. USE was trained in an unsupervised manner using data from a variety of web sources, including Wikipedia, web news, web question-answer pages and discussion forums, and augmented with supervised data from the Stanford Natural Language Inference (SNLI) corpus [Bowman et al., 2015]. The goal of USE and much follow-up work was to compute embeddings of segments of texts (sentences, paragraphs, etc.) for similarity comparisons.

Work on BERT-based dense representations for similarity comparisons emerged in 2019 from a few sources. To our knowledge, the earliest paper is by Humeau et al. [2019], dating from April 2019. We use the bi-encoder vs. cross-encoder terminology that they introduced. Although the work examined retrieval tasks, the setup was limited in scope (see Section 5.5.1 for more details). Several roughly contemporaneous papers appeared shortly thereafter. Sentence-BERT [Reimers and Gurevych, 2019] applied the bi-encoder design to a number of sentence-similarity tasks. At the same time, Barkan et al. [2020] investigated how well a BERT-based cross-encoder could be distilled into a BERT-based bi-encoder, also in the context of sentence-similarity tasks.¹⁷⁷ However, neither Reimers and Gurevych [2019] nor Barkan et al. [2020] explicitly examined retrieval tasks.

In terms of explicitly applying transformer-based bi-encoders to retrieval tasks, we believe precedence goes to Lee et al. [2019b].¹⁷⁸ However, instead of direct retrieval supervision using labeled data, they elected to focus on pretraining using weak supervision techniques derived from the Inverse Cloze Task (ICT) [Taylor, 1953]. Related work by Guu et al. [2020] folded dense retrieval directly into the pretraining regime. As later demonstrated by Karpukhin et al. [2020b] on some of the same question-answering benchmarks, these approaches did not appear to be as effective as direct retrieval supervision: Lee et al. [2019b] reported uneven gains over previous approaches based on BM25 + BERT such as BERTserini [Yang et al., 2019c] and the techniques proposed by Guu et al. [2020] appeared to be more complex, more computationally expensive, and less effective. However, as explained by Kenton Lee (based on personal communications), these two papers aimed to tackle a different problem, a setup where annotated data for direct supervision was unavailable, and thus required different solutions. For this reason, it might not be fair to only compare these techniques in terms of effectiveness.

Shortly thereafter, Yang et al. [2019a] proposed PairwiseBERT, which applied bi-encoders to align cross-lingual entities in knowledge graphs by comparing textual descriptions of those entities; this was formulated as a cross-lingual ranking problem. Also contemporaneous was the “two-tower

¹⁷⁷The first arXiv submission of Humeau et al. [2019] unambiguously pre-dated Sentence-BERT, as the latter cites the former. However, Humeau et al.’s original arXiv paper did not appear in a peer-reviewed venue until April 2020, at ICLR [Humeau et al., 2020]. The arXiv versions of Reimers and Gurevych [2019] and Barkan et al. [2020] appeared within two weeks of each other in August 2019.

¹⁷⁸As an interesting historical side note, similar ideas (but not using transformers) date back at least a decade [Yih et al., 2011], and arguably even further back in the context of supervised dimensionality reduction techniques [Yu et al., 2006]. What’s even more remarkable is that some of the co-authors of Yih et al. [2011] are also co-authors on recent dense retrieval papers, which suggests that these ideas had been “brewing” for many years, and finally, with pretrained transformers, the “technical machinery” finally “caught up” to enable the successful execution of much older ideas and insights. See additional discussion in Section 6 where we wonder if everything’s a remix.

Source: Lin et al. (2021), Figure 23. Cross-encoder (top) requires inference for all pairs. Bi-encoder (bottom) allows pre-computation of document representations

Bi-encoder encodes queries and documents **independently**:

$$\mathbf{q} = E_q(\text{query}), \quad \mathbf{d} = E_d(\text{document})$$

$$\text{score}(q, d) = \text{sim}(\mathbf{q}, \mathbf{d})$$

The similarity function `sim` uses **dot product** or **cosine similarity**

Key advantages:

Pre-compute document vectors: Compute and store vectors for all documents offline

Only encode the query at search time: One BERT inference + vector search

No need for pairwise document comparison -> Search possible in **milliseconds**

Bi-encoder Retrieval Pipeline

Offline: Document Encoding

Encode all documents into dense vectors with BERT -> Build index



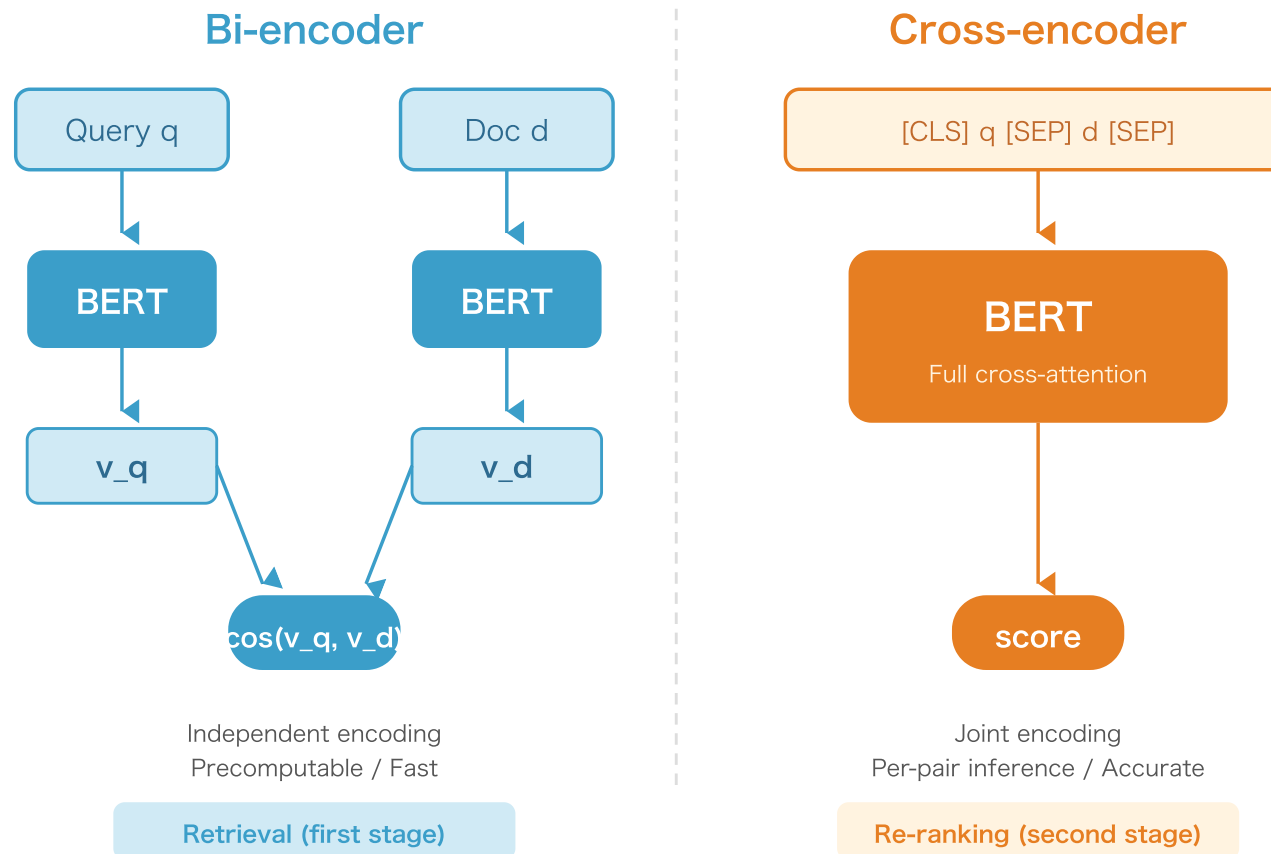
Online: Query Encoding

Encode query into a dense vector with BERT (single inference)



Nearest Neighbor Search (ANN)

Fast retrieval of document vectors closest to the query vector



Design philosophy difference: Cross-encoder prioritizes accuracy, Bi-encoder enables pre-computation + fast retrieval. In practice, a multi-stage architecture of **Bi-encoder for retrieval** -> **Cross-encoder for reranking** is common

Nearest Neighbor Search

Approximate Nearest Neighbor (ANN) Search

The problem of finding the **most similar document** from dense vectors obtained by a Bi-encoder:

$$d^* = \arg \max_{d \in \mathcal{D}} \text{sim}(\mathbf{q}, \mathbf{d})$$

Exact brute-force search:

Compute cosine similarity/dot product with all documents

$O(N \cdot D)$: Number of documents N x Dimensionality D

8.8 million documents x 768 dimensions -> Takes several seconds

Approximate Nearest Neighbor (ANN) search is needed:

Does not guarantee exact solutions but searches in **milliseconds**

In practice, recall of top documents can exceed 95%

Exact Search vs Approximate Search

Brute-force

Compute scores with all documents -> **Exact**

Complexity: $O(N \times D)$

8.8M documents: **Several seconds**

ANN (Approximate Nearest Neighbor)

Partition space / build graph to narrow candidates -> **Fast**

Complexity: $O(\log N)$ to $O(N^{0.5})$

8.8M documents: **Milliseconds**

HNSW (Hierarchical Navigable Small World) is an ANN algorithm that uses graph structures

Basic idea:

Build a graph where each vector is a node with edges connecting nearby vectors

Hierarchical structure: Upper layers have coarse connections (long-distance jumps), lower layers have dense connections

Search flow:

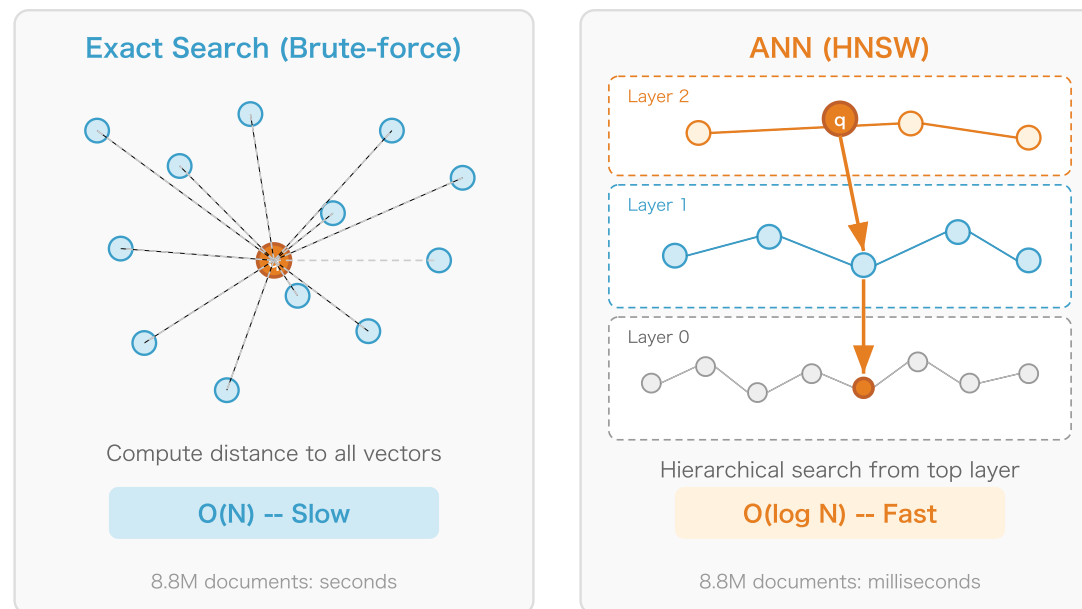
1. Find approximate neighbors in the topmost layer
2. Gradually refine through lower layers
3. Return nearest neighbors at the bottom layer

Comparison with IVF:

HNSW: Higher memory usage but **higher accuracy**

IVF+PQ: Better memory efficiency but **slightly**

Exact Search vs ANN (HNSW)



Left: Compute distances to all vectors ($O(N)$). Right: HNSW searches hierarchically through a graph ($O(\log N)$)

Sentence-BERT

Sentence Embeddings via Siamese Networks

Using BERT directly for sentence similarity computation requires a **Cross-encoder for all pairs**:

Finding the most similar sentence pair among 10,000 sentences

$\binom{10000}{2} \approx 50$ million pairs

~65 hours (V100 GPU)

What about using BERT's [CLS] token directly as a sentence vector?

Experiments showed performance **worse** than GloVe averaged vectors

BERT is trained as a Cross-encoder, and its standalone sentence representation is not optimized

-> A Bi-encoder that can independently produce dense vectors for sentences is needed

Why BERT's [CLS] Is Not Sufficient

Spearman Correlation on STS Benchmark

GloVe average: **58.02**

BERT [CLS] (no fine-tuning): **29.19**

BERT average pooling (no FT): **46.35**

Sentence-BERT: 84.67

Pre-trained BERT's [CLS] does not capture semantic textual similarity. Sentence-BERT explicitly learns sentence representations through a Siamese Network

Siamese Network: Uses two BERTs with shared weights to independently encode two sentences

Structure:

1. Input sentence A and sentence B into separate BERTs
2. **Pool** the output tokens of each BERT (mean pooling is best)
3. Compute the objective function using the resulting sentence vectors $\mathbf{u}$, $\mathbf{v}$

Comparison of pooling strategies:

[CLS] token: Lowest performance

Mean pooling: Average of all tokens -> **Best**

Max pooling: Maximum value in each dimension

- For regression tasks, mean squared loss between the ground truth and the cosine similarity of the two sentence embeddings $\mathbf{u}$ and $\mathbf{v}$ is used.

Reimers and Gurevych [2019] additionally proposed a triplet loss structure, which we do not cover here because it was only applied to one of the evaluation datasets.

At inference time, the trained encoder η is applied to both sentences, producing sentence vectors $\mathbf{u}$ and $\mathbf{v}$. The cosine similarity between these two vectors is directly interpreted as a similarity score; this is shown in Figure 23, right. That is, in our terminology, $\phi(\mathbf{u}, \mathbf{v}) = \cos(\mathbf{u}, \mathbf{v})$. This provides the answer to the comparison problem discussed in Section 5.1.

Sentence-BERT was evaluated in three different ways for textual similarity tasks:

- *Untrained.* BERT (or RoBERTa) can be directly applied “out of the box” for semantic similarity computation.
- *Fine-tuned on out-of-domain datasets.* Sentence-BERT was fine-tuned on a combination of the SNLI and Multi-Genre NLI datasets [Bowman et al., 2015, Williams et al., 2018]. The trained model was then evaluated on the Semantic Textual Similarity (STS) benchmark [Cer et al., 2017].
- *Fine-tuned on in-domain datasets.* Sentence-BERT was first fine-tuned on the SNLI and Multi-Genre NLI datasets (per above), then further fine-tuned on the training set of the STS benchmark before evaluation on its test set. This is similar to the multi-step fine-tuning approaches discussed in Section 3.2.4.

Below, we present a few highlights summarizing experimental results, but refer readers to the authors’ original paper for details. Sentence-BERT was primarily evaluated on sentence similarity tasks, not actual retrieval tasks, and since we do not present results on these tasks elsewhere in this survey, reporting evaluation figures here would be of limited use without points of comparison. Nevertheless, there are a number of interesting findings worth discussing:

- Without any fine-tuning, average pooling of BERT’s contextual representations appears to be worse than average pooling of static GloVe embeddings, based on standard metrics for semantic similarity datasets. Using the [CLS] token was even worse than average pooling, suggesting that it is unable to serve as a good representation “out of the box” (that is, without fine-tuning on task-specific data).
- Not surprisingly, out-of-domain fine-tuning leads to large gains on the STS benchmark over the untrained condition. Also as expected, further in-domain fine-tuning provides an additional boost in effectiveness, consistent with the multi-step fine-tuning approaches discussed in Section 3.2.4. In this setting, although the bi-encoder remained consistently worse than the cross-encoder, in some cases the differences were relatively modest.
- Ablation studies showed that with fine-tuning, average pooling was the most effective design for η , slightly better than max pooling or using the [CLS] token. Although the effectiveness of the [CLS] token was quite low “out of the box” (see above), after fine-tuning, it was only slightly worse than average pooling.
- For classification tasks, an interesting finding is the necessity of including $|\mathbf{u} - \mathbf{v}|$ in the input to the softmax classifier (see above). If the input to the softmax omits $|\mathbf{u} - \mathbf{v}|$, effectiveness drops substantially.

Closely related to Sentence-BERT, the contemporaneous work of Barkan et al. [2020] investigated how well a BERT-based cross-encoder can be distilled into a BERT-based bi-encoder for sentence similarity tasks. To do so, the authors trained a BERT_{large} cross-encoder to perform a specific task and then distilled the model into a BERT_{large} bi-encoder that produces a dense representation of its input by average pooling the outputs of its final four transformer layers. The experimental results were consistent with the same general findings in Sentence-BERT: After distillation for a specific task, the bi-encoder student performs competitively but remains consistently less effective than the cross-encoder teacher. However, as expected, the bi-encoder is significantly more efficient.

Takeaway Lessons. Sentence-BERT provides a good overview of the basic design of bi-encoders, but its focus was on textual similarity and not ranking. For a range of sentence similarity tasks, the

142

Source: Lin et al. (2021), Figure 24. Two sentences are independently encoded by weight-shared BERTs, and sentence vectors are generated through pooling

Results on STS (Semantic Textual Similarity):

Approaches Cross-encoder BERT accuracy while being **thousands of times faster**

Significantly outperforms GloVe and BERT [CLS]

Inference speed: Comparing 10,000 sentence pairs — BERT: 65 hours -> **SBERT: ~5 seconds**

Significance of Sentence-BERT:

First method to extract **high-quality sentence vectors** from BERT

Foundation technology for sentence similarity search, clustering, and semantic search

Paved the way for Dense Retrieval

Accuracy and Speed Comparison

STS Benchmark (Spearman Correlation)

SBERT-NLI-base	84.67
BERT-base [CLS]	29.19
GloVe average	58.02

DPR — Dense Passage Retrieval

Large-scale Document Retrieval with Bi-encoders

DPR (Dense Passage Retrieval) is a Bi-encoder-based Dense Retrieval model designed for Open-domain QA

Differences from Sentence-BERT:

Sentence-BERT: Learns sentence **similarity** (NLI/STS data)

DPR: Directly learns to **retrieve relevant documents** for a query

Architecture:

Query encoder E_Q : BERT-base

Document encoder E_P : BERT-base (separate parameters)

[CLS] token from each encoder used as the sentence vector

Similarity: **Dot product** $\text{sim}(q, p) = E_Q(q)^\top E_P(p)$

the answer. With approach (3), the idea of training with in-batch negatives can be traced back to at least Henderson et al. [2017], who also applied the technique to train a bi-encoder for retrieval, albeit with simple feedforward networks over n -grams instead of transformers.

Empirically, approach (3) proved to be the most effective, and it is efficient as well since the negative examples are already present in the batch during training. Furthermore, effectiveness increases as the batch size grows, and thus the quality of the encoders improves as we are able to devote more computational resources during training. We refer interested readers to the original paper for details regarding the exact experimental settings and results of contrastive experiments that examine the impact of different negative sampling approaches.

DPR was evaluated on a number of standard question answering datasets in the so-called “open-domain” (i.e., retrieval-based) setting, where the task is to extract answers from a large corpus of documents—in this case, a snapshot of English Wikipedia. Following standard experimental settings, passages were constructed from Wikipedia articles by taking 100-word segments of text; these formed the units of retrieval and served as inputs to E_Q . The five QA datasets used were Natural Questions [Kwiatkowski et al., 2019], TriviaQA [Joshi et al., 2017], WebQuestions [Berant et al., 2013], CuratedTREC [Baudis and Sedivy, 2015], and SQuAD [Rajpurkar et al., 2016].

Here, we are only concerned with retrieval effectiveness, as opposed to end-to-end QA effectiveness. The commonly accepted metric for this task is top- k accuracy, $k \in \{20, 100\}$, which measures the fraction of questions for which the retriever returns at least one correct answer. This is akin to measuring recall in a multi-stage ranking architecture (see Section 3.4): in a pipeline design, these metrics quantify the upper bound effectiveness of downstream components. In the case of question answering, if the retriever doesn’t return candidate texts containing answers, there’s no way for a downstream reader to recover. Note that in the NLP community, metrics are often reported in “points”, i.e., values are multiplied by 100, so 0.629 is shown as 62.9.

Instead of directly reporting results from Karpukhin et al. [2020b], we share results from Ma et al. [2021c], which is a replication study of the original paper. Ma et al. were able to successfully replicate the dense retrieval results and obtain scores that were very close to those in the original paper (in most cases, within a tenth of a point). However, their experiments led to a substantive contrary finding: according to the original paper, there is little to be gained from a hybrid technique combining DPR (dense) with BM25 (sparse) results via linear combination. In some cases, DPR alone was more effective than combining DPR with BM25, and even if the hybrid achieved a higher score, the improvements were marginal at best. The experiments of Ma et al., however, reported higher BM25 scores than the original paper.¹⁴⁰ This, in turn, led to higher effectiveness for the hybrid technique, and thus Ma et al. concluded that DPR + BM25 was more effective than DPR alone. In other words, dense-sparse hybrids appear to offer benefits over dense retrieval alone.

Table 38 shows the DPR replication results, copied from Ma et al. [2021c]. The authors applied paired t -tests to determine the statistical significance of the differences ($p < 0.01$) with the Bonferroni correction as appropriate. The symbol $\dagger$ on a BM25 result indicates that the effectiveness difference vs. DPR is significant; the symbol $\ddagger$ indicates that the hybrid technique is significantly better than BM25 (for SQuAD) or DPR (for all remaining collections). We see that in four of the five datasets, dense retrieval alone (DPR) is more effective than sparse retrieval (BM25); in these cases, the differences are statistically significant for both top-20 and top-100 accuracy.¹⁴¹ Ma et al. [2021c] experimented with two different approaches for combining DPR with BM25 scores; as there were no significant differences between the two, we report the technique they called Hybrid_{gem} (see paper for details). According to their results, in most cases, the dense-sparse hybrid was more effective than BM25 (for SQuAD) or DPR (for all remaining collections). The improvements were statistically significant in nearly all cases.

Building on the basic bi-encoder design, Xiong et al. [2021] made the observation that non-relevant texts ranked highly by an exact match method such as BM25 are likely to be different from non-relevant texts ranked highly by a BERT-based bi-encoder. Thus, selecting negative examples from

¹⁴⁰This finding has been confirmed by the original authors (personal communication).

¹⁴¹The exception appears to be SQuAD, where BM25 effectiveness is higher, likely due to two reasons: First, the dataset was created from only a few hundred Wikipedia articles, and thus the distribution of the training examples is highly biased. Second, questions were created by human annotators based on the articles, thus leading to question formulations with high lexical overlap, giving an unnatural and unfair advantage to an exact match technique like BM25.

Source: Lin et al. (2021), Figure 25. DPR uses two independent BERT encoders. Query and document have separate parameters

The most important technique in DPR training is **in-batch negatives**

Basic idea:

A batch contains B (query, positive document) pairs

For each query, **positive documents of other queries** are used as negatives

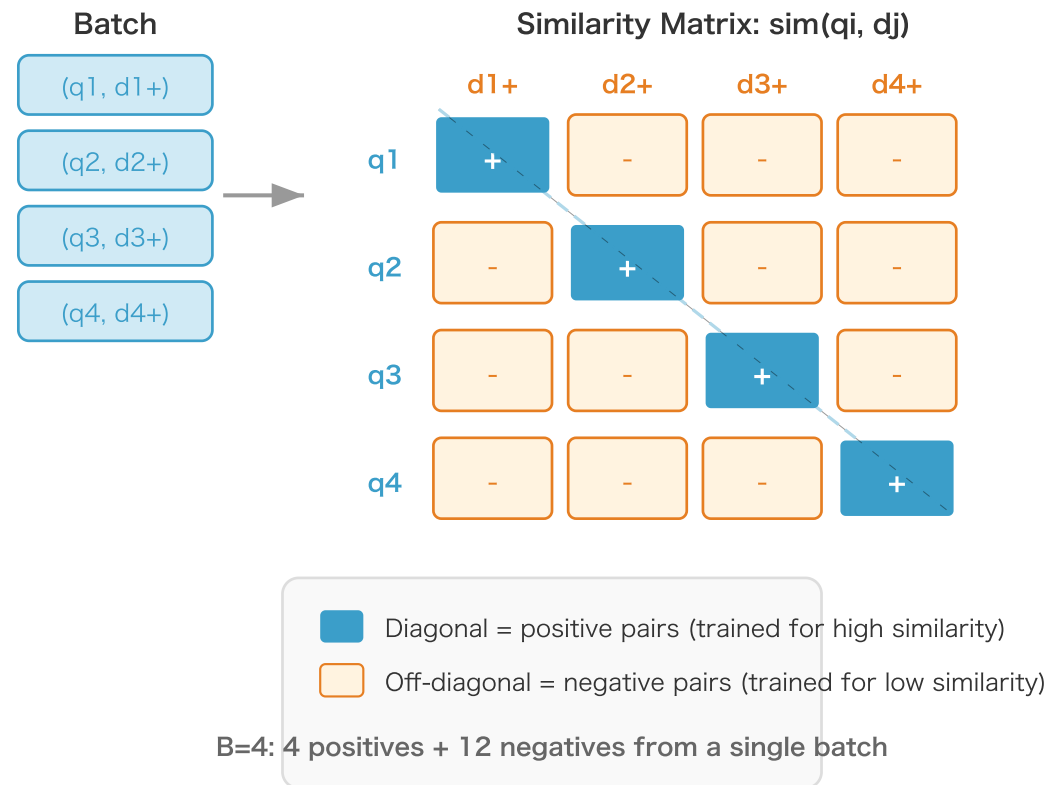
Obtain $B - 1$ **negatives** without additional negative encoding

Loss function (Negative Log-Likelihood):

$$\mathcal{L} = -\log \frac{e^{\text{sim}(q_i, p_i^+)}}{e^{\text{sim}(q_i, p_i^+)} + \sum_{j \neq i} e^{\text{sim}(q_i, p_j^+)}}$$

Larger batch sizes yield more negatives and improve performance

DPR: In-batch Negatives Training



Diagonal elements are positives (blue), off-diagonal elements are negatives (orange). With $B=128$, each query obtains 127 negatives

In-batch negatives alone are insufficient — random negatives are "too easy"

Hard Negatives are:

Documents that **seem relevant but are actually non-relevant**

e.g., documents scoring high with BM25 but judged non-relevant in qrels

Hard Negative strategy in DPR:

Use top BM25 results (excluding the correct document) as Hard Negatives

Each query has 1 Gold (correct) + 1 BM25 hard negative + in-batch negatives

Effect (Natural Questions dataset, Top-20 Recall):

Random negatives only: 78.4%

Adding BM25 hard negatives: **79.8%**

Hard negatives from another encoder using Gold

Levels of Negative "Difficulty"

Easy Negatives

Randomly sampled documents. Low relevance to the query, easily distinguishable by the model

BM25 Hard Negatives

High BM25 score but non-relevant. Lexically similar but semantically mismatched

DPR Hard Negatives

High DPR score but non-relevant. The examples the model struggles with most

Top-k retrieval accuracy (Recall) on Natural Questions:

DPR achieves retrieval performance **significantly surpassing BM25**

Top-20 Recall:

BM25: 59.1%

DPR (single dataset training): **79.4%**

DPR (multi-dataset training): **79.4%**

Top-100 Recall:

BM25: 73.7%

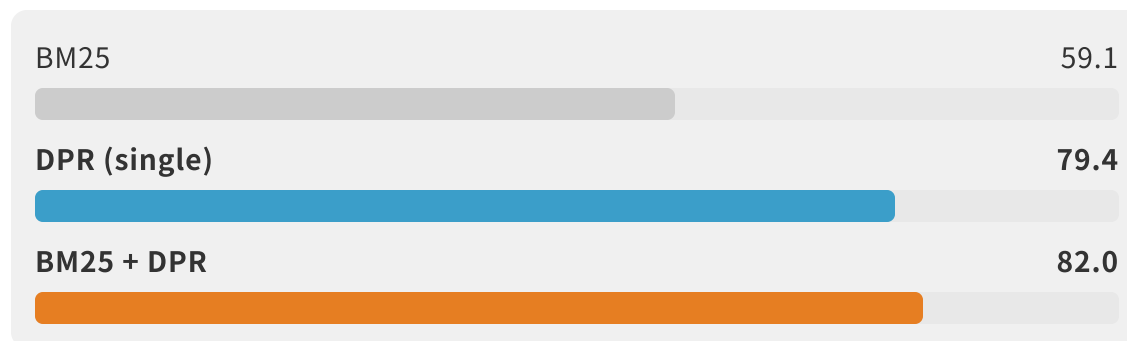
DPR: **86.0%**

Combining BM25 + DPR is also effective:

Linear interpolation of scores achieves **82.0%**
(Top-20)

Complementarity of lexical matching and semantic matching

Natural Questions: Top-20 Recall



Source: Karpukhin et al. (2020), Table 2. DPR overcomes vocabulary mismatch and surpasses BM25 by over 20 points

ANCE

Approximate Nearest Neighbor Negative Contrastive Estimation

ANCE (Approximate Nearest Neighbor Negative Contrastive Estimation) obtains hard negatives from **the model's own retrieval results**

Algorithm:

1. Encode all documents with the current model -> Build ANN index
2. Run ANN search for each query, collect top non-relevant documents as hard negatives
3. Train the model for several steps using the collected hard negatives
4. **Periodically** update the index (return to step 1)

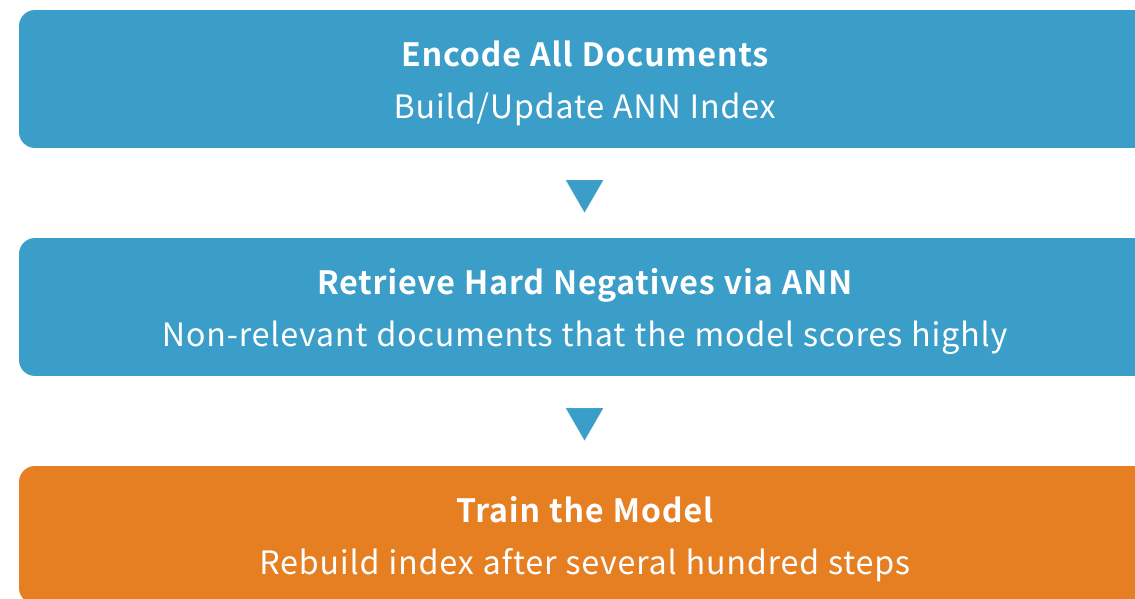
Asynchronous updates:

Index reconstruction is costly

Update the index **every several hundred to thousand steps**

Negatives from an older index are still effective

ANCE Training Loop



Index update frequency is a trade-off: frequent -> accurate negatives but costly, infrequent -> low cost but stale negatives

ColBERT

Efficient Matching via Late Interaction

Bi-encoders compress an entire document into a **single vector**. This has inherent limitations:

Information bottleneck:

Packing the diverse content of a long document into 768 dimensions

Difficult to capture partial matches like "restaurants in Tokyo" vs "tourist spots in Tokyo"

Cause of accuracy gap with Cross-encoders:

Cross-encoder: Captures **token-level** interactions between query and document

Bi-encoder: Only captures **sentence-level** similarity between query and document

Late Interaction as a solution:

Represent documents with **multiple token vectors**

Perform token-level matching at search time

Comparison of Representational Power

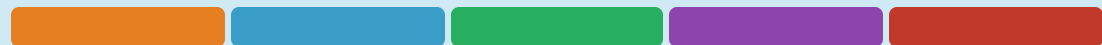
Bi-encoder: Single Vector



Entire document -> 1 vector

Information is compressed, fine-grained meaning is lost

ColBERT: Token Vector Sets



Each token retains its own vector -> Rich representation

ColBERT (Contextualized Late Interaction over BERT) preserves token-level representations while enabling pre-computation

Encoding:

Query: $\mathbf{Q} = \text{BERT}(q) \in \mathbb{R}^{L_q \times d}$

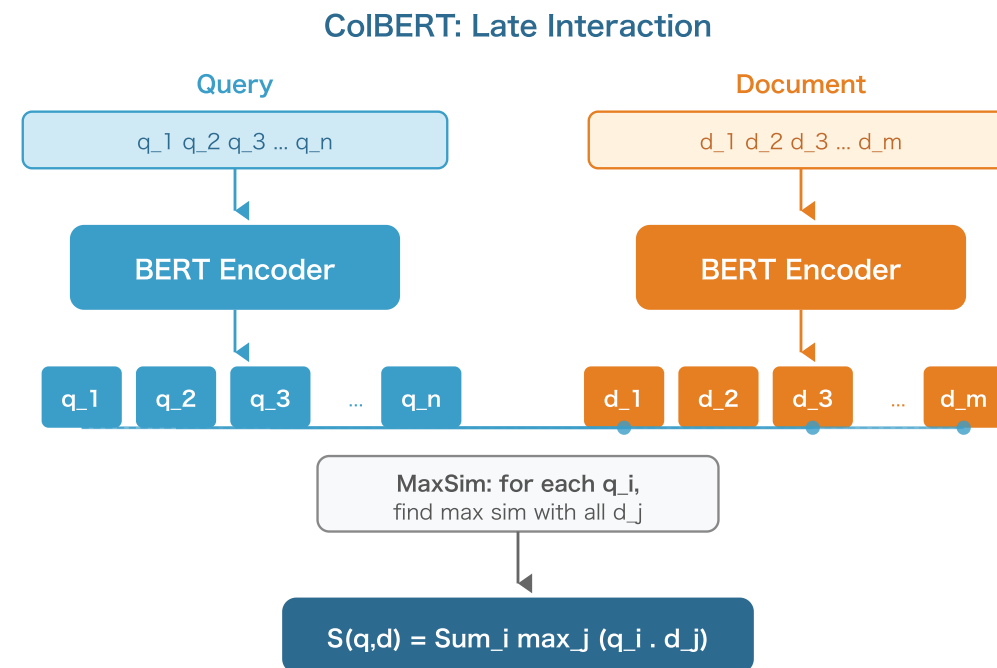
Document: $\mathbf{D} = \text{BERT}(d) \in \mathbb{R}^{L_d \times d}$

Each token output is projected to $d = 128$ dimensions through a linear layer

MaxSim scoring:

$$S(q, d) = \sum_{i=1}^{L_q} \max_{j=1}^{L_d} \mathbf{Q}_i \cdot \mathbf{D}_j$$

Sum the score with the **most similar document token** for each query token



Token embeddings precomputed offline; MaxSim computed at query time

ColBERT's Late Interaction (based on Lin et al. (2021), Figure 27).
Document token vectors can be pre-computed

Intuitive understanding of MaxSim:

Each query token finds the **semantically closest position** in the document

Example: Query "best time for autumn leaves in Kyoto"

"Kyoto" -> High similarity with "Kyoto", "Kansai", "Arashiyama" in the document

"autumn leaves" -> High similarity with "autumn leaves", "foliage", "fall colors" in the document

"best time" -> High similarity with "season", "November", "timing" in the document

Difference from Cross-encoder:

Cross-encoder: Full attention across all tokens

ColBERT: Attention within each encoder ->

Aggregation via MaxSim

Difference from Bi-encoder:

ColBERT: MaxSim Computation

Query	Document				Similarity Matrix				
	d1	d2	d3	d4	d1	d2	d3	d4	Max
q1					0.9	0.3	0.2	0.1	0.9
q2					0.2	0.4	0.8	0.5	0.8
q3					0.1	0.7	0.3	0.6	0.7
									Sum

MaxSim Scoring

$$S(q, d) = \text{MaxSim}(q1, D) + \text{MaxSim}(q2, D) + \text{MaxSim}(q3, D)$$

$$= 0.9 + 0.8 + 0.7 = 2.4$$

For each query token, select the most similar token in the document

$$\text{MaxSim}(q_i, D) = \max \text{ over } j \text{ of } \text{sim}(q_i, d_j)$$

Captures partial matches: fine-grained matching impossible with Bi-encoder

Sum of maximum similarity for each query token. Captures partial matches. Numbers in the conceptual diagram are for illustration

Results on MS MARCO Passage Ranking:

MRR@10: 0.349 (re-ranking), 0.360 (end-to-end)

Accuracy comparable to BERT re-ranker (Cross-encoder)

Significantly faster than exhaustive Cross-encoder comparison (tens to hundreds of times faster depending on configuration)

Results on TREC DL 2019:

BM25 + ColBERT re-ranking achieves **nDCG@10: 0.706**

Significantly outperforms BM25 alone (0.506)

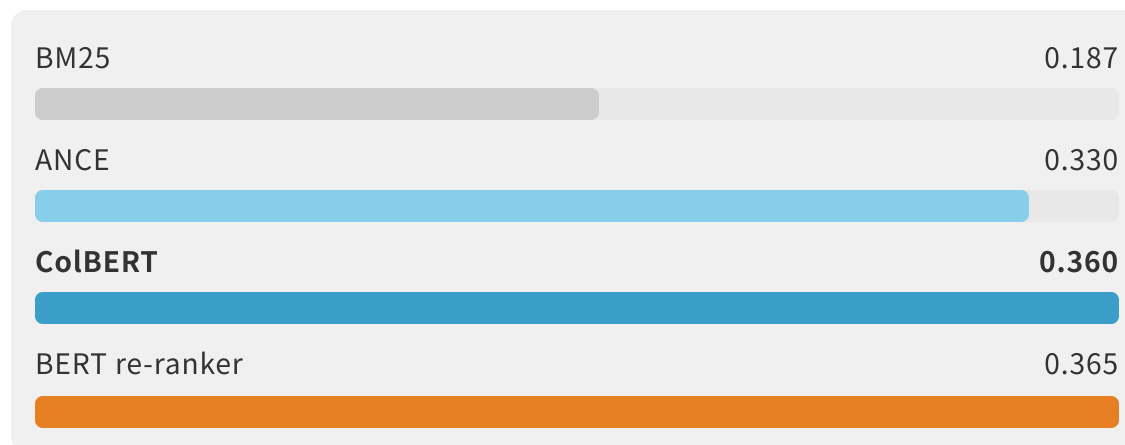
Improvements in ColBERTv2 [Santhanam et al., 2022]:

Residual compression: Quantize token vectors

6-10x storage reduction

Practical storage size while maintaining accuracy

MS MARCO: MRR@10 Comparison



Source: Khattab and Zaharia (2020). ColBERT achieves accuracy nearly equivalent to Cross-encoder (BERT re-ranker)

Design Space of Bi-encoders

Method Comparison and Poly-encoder

Method	Interaction	Doc Representation	Negative Strategy	Search Speed	Storage
Cross-encoder	Full Attention	Not pre-computable	BM25 negatives	Very slow	-
Sentence-BERT	None (dot product)	1 vector	NLI/Triplet	Fastest	Small
DPR	None (dot product)	1 vector	In-batch + BM25	Fastest	Small
ANCE	None (dot product)	1 vector	ANN Hard Neg	Fastest	Small
ColBERT	Late (MaxSim)	Token vector sets	Pairwise	Fast	Large
Poly-encoder	Attention codes	1 vector	In-batch	Fast	Small

Key points of the design space: (1) Depth of interaction (None -> Late -> Full), (2) Granularity of document representation (1 vector -> Token vector sets), (3) Quality of negatives (Random -> BM25 -> Model's own). These trade-offs determine the method

1. **Bi-encoder:** Encodes queries and documents independently and retrieves by dense vector similarity. Pre-computation of document vectors enables large-scale retrieval that is impossible with Cross-encoders
2. **DPR / ANCE:** Improve Bi-encoder training through in-batch negatives and ANN-based hard negative mining. Negative quality significantly impacts performance
3. **ColBERT:** A Late Interaction model that retains token-level vectors and aggregates via MaxSim. Achieves intermediate accuracy and speed between Bi-encoders and Cross-encoders
4. **Design space of Dense Retrieval:** Methods are determined by trade-offs in interaction depth, document representation granularity, and negative strategy. In production, hybrid approaches with BM25 are common

Preview of Next Lecture: Learned Sparse Retrieval

SPLADE: Learned sparse representations and vocabulary expansion

Comprehensive comparison of Sparse vs Dense vs Late Interaction

Hybrid search and overall summary